

NumPy + SciPy — Reference Card

arrays · indexing · broadcasting · axis · linalg · RNG · ufuncs + scipy stats/optimize/sparse. Stack: numpy ≥2.0 · scipy ≥1.14 · Python ≥3.10 · Modern: np.random.default_rng(), explicit axis

◆ Master pattern (every numpy script)

- 1. **RNG** — rng = np.random.default_rng(seed) (not np.random.seed)
- 2. **Create** — np.zeros / arange / linspace / from list. Declare dtype.
- 3. **Reshape** — .reshape(-1, n) for -1 auto-dim
- 4. **Index / mask** — basic slice, boolean, fancy; know view-vs-copy
- 5. **Vectorise** — ufuncs; specify axis= on reductions
- 6. **Linalg** — solve not inv @; specific factorisations

Mental model: think in ARRAYS + AXES, not loops. axis=0 collapses rows (result has cols); axis=1 collapses cols (result has rows).

◆ Starter · reproducible skeleton

```
import numpy as np
from numpy.typing import NDArray
```

```
SEED = 42
rng = np.random.default_rng(SEED)           # canonical RNG
(not np.random.seed)
```

```
# Explicit dtypes – avoid float64/int64 default surprises
X = rng.standard_normal((1000, 5)).astype(np.float32)
y = rng.integers(0, 2, size=1000, dtype=np.int32)
```

```
# Or with schema:
def sample(n: int) -> tuple[NDArray[np.float32],
                             NDArray[np.int32]]:
    X = rng.standard_normal((n, 5), dtype=np.float32)
    y = rng.integers(0, 2, size=n, dtype=np.int32)
    return X, y
```

```
# Debug utilities
np.set_printoptions(precision=4, suppress=True, linewidth=140)
np.testing.assert_allclose(a, b, rtol=1e-5)           # test near-equal
```

● Array creation

Task	Idiom
From list	np.array([1,2,3], dtype=np.int32)
Zeros / ones / full	np.zeros((3,4)) · np.ones · np.full((3,4), 7.0)
Like another	np.zeros_like(x)
Identity	np.eye(n)
Range	np.arange(0, 10, 0.5)
Linspace	np.linspace(0, 1, 51) — INCLUSIVE endpoint
Log space	np.logspace(-4, 0, 20)
Empty (fast, uninit)	np.empty((n, m)) — YOU must fill it
Meshgrid (2D eval)	X, Y = np.meshgrid(x, y, indexing="ij")

Trap: np.empty returns UNINITIALISED memory. Reading before writing = garbage bytes.

● Indexing / slicing

```
a = np.arange(24).reshape(4, 6)
a[1, 2]           # single element (int)
a[1]             # row 1 (1D)
a[:, 2]          # col 2 (1D)
a[1:3, 2:5]      # sub-block (view!)
a[:, :2]         # every other row
a[::-1]          # reversed
a[[0, 2], :]     # fancy: rows 0 and 2 (copy!)
a[a > 10]        # boolean mask (copy, 1D)
a[np.ix_([0,2], [1,3])] # cross-product indexing
```

Type	Returns
Basic slice (:)	VIEW — mutations propagate
Fancy (list/int-array)	COPY — safe to modify
Boolean mask	COPY — safe to modify

View trap: a[1:3] = 0 mutates a too. Check via b.base is a. To force copy: a[1:3].copy().

▲ Broadcasting (the key insight)

Rule: align shapes from the RIGHT. Pad missing left dims with 1.
Compatible if each dim is EQUAL or ONE of them is 1.

```
a = np.zeros((3, 5))           # (3, 5)
b = np.arange(5)               # (5,)   → (1, 5) → broadcasts to
(3, 5)                         # OK
a + b
```

```
col = np.arange(3).reshape(3, 1) # (3, 1)
row = np.arange(5)               # (5,)   → (1, 5)
col + row                       # (3, 5) — outer sum
```

```
# Add a trailing dim via None / np.newaxis for explicit intent
X = np.zeros((100, 5))
mean = X.mean(axis=0, keepdims=True) # (1, 5) — safer than
mean.shape (5,)                     # broadcasts (100, 5) -
(1, 5) = (100, 5)
```

Keep keepdims=True on reductions when you plan to broadcast back — avoids a lot of .reshape.

▲ Axis reductions (always specify axis=)

Operation	Idiom
sum / mean / std / var	x.sum(axis=0)
min / max / argmin / argmax	x.argmax(axis=1)
any / all	mask.any(axis=1)
cumsum / cumprod	x.cumsum(axis=0)
Percentile / quantile	np.quantile(x, q=[0.05, 0.5, 0.95], axis=0)
Median / count_nonzero	np.median(x, axis=0)
Weighted average	np.average(x, weights=w, axis=0)

Sensitive to NaN → use nan-aware variants
np.nansum(x, axis=0); np.nanmean; np.nanstd; np.nanquantile
ddof= for sample std/var (default ddof=0 = population)
x.std(axis=0, ddof=1) # sample std

▲ ufuncs + where / select

np.add, subtract, multiply, divide, power, sqrt, log, log1p, exp, expm1
np.sin, cos, tan, arctan2, sinh, tanh
np.abs, sign, round, floor, ceil, trunc, clip(x, lo, hi)
np.maximum(a, b), np.minimum(a, b) # element-wise pairs
np.logical_and(m1, m2), logical_or, logical_not
np.isnan, isinf, isfinite

```
# Conditional selection
np.where(x > 0, x, 0)           # like ternary: cond,
if true, if false
np.where(x > 0)                # returns indices
(tuple)
```

```
# Multi-way
np.select([m1, m2, m3], [v1, v2, v3], default=0)
```

```
# In-place ufunc output → avoid allocations in tight loops
np.add(a, b, out=a)
np.exp(a, out=a)               # in-place exp
```

ufuncs are **always vectorised**, always release the GIL. Prefer them over Python loops.

◆ RNG (default rng, not seed)

```
rng = np.random.default_rng(42)           # ← canonical entry
point
# Reproducible child streams (for parallel workers):
child = rng.spawn(4)                     # 4 independent RNGs
```

Top methods (all return arrays; size= for shape)

```
rng.random(size=(3, 4))             # uniform [0, 1)
rng.standard_normal(size=1000)      # N(0, 1)
rng.normal(loc=0, scale=1, size=100) # N(loc, scale)
rng.uniform(low=-1, high=1, size=100) # U[low, high)
rng.integers(low=0, high=10, size=5) # ints [low, high) —
EXCLUSIVE high
rng.integers(low=0, high=10, size=5, endpoint=True) #
inclusive high
rng.choice([1,2,3], size=5, p=[.5,.3,.2], replace=False)
rng.permutation(np.arange(10))      # shuffle out-of-place
rng.multinomial(n=100, pvals=[.3,.3,.4])
rng.multivariate_normal(mean=mu, cov=Sigma, size=1000)
```

Legacy: np.random.seed(42); np.random.rand(...) — deprecated.
Global RNG state = non-reproducible in parallel code.

● Reshape / transpose / concat

```
a.reshape(3, -1)                 # -1 = infer from total
a.reshape(-1); a.ravel()         # flatten (ravel may copy)
a.T                              # transpose axes (view)
np.transpose(a, axes=(2, 0, 1))  # arbitrary axis reorder
a[np.newaxis, :]; a[:, None]    # add axis (None ≡ np.newaxis)
a.squeeze()                     # drop size-1 dims
```

```
np.concatenate([a, b], axis=0)    # existing axis
np.stack([a, b], axis=0)          # NEW axis
np.split(a, 3, axis=0)            # exact split; array_split
allows uneven
np.tile(a, (3, 1)); np.repeat(a, 3, axis=0)
```

▲ Linear algebra

```
A @ B                             # matmul (prefer @ over np.dot
for 2D)
np.dot(a, b)                      # 1D dot; 2D matmul (equiv to @)
np.linalg.norm(x, ord=2, axis=0)  # L2 by column
```

```
# Solve Ax = b — NEVER compute inv(A) @ b explicitly
x = np.linalg.solve(A, b)         # canonical
x = np.linalg.lstsq(A, b, rcond=None)[0] # least-squares
(over/underdetermined)
```

```
# Decompositions
U, s, Vt = np.linalg.svd(A, full_matrices=False) # SVD
(economy form)
L = np.linalg.cholesky(A)         # A = L LT
(PSD only)
Q, R = np.linalg.qr(A)            # QR
w, V = np.linalg.eig(A)           #
eigenvalues + right eigenvectors
w, V = np.linalg.eigh(A)          #
SYMMETRIC A — faster + stable
```

```
# Matrix inverse — rarely needed; use solve instead
np.linalg.inv(A)                  # explicit inverse
np.linalg.pinv(A)                 # Moore-Penrose pseudo-inverse
```

```
# Determinant / trace
np.linalg.det(A); np.trace(A)
```

```
# Batched linalg (broadcasting on last two dims)
np.linalg.solve(A_stack, b_stack) # A_stack.shape = (B, n, n)
→ x.shape (B, n)
```

```
# einsum — the daily-driver for batched / non-obvious linalg
np.einsum("ij,jk->ik", A, B)     # ≡ A @ B
np.einsum("ii->i", A)            # diagonal
np.einsum("ii->", A)              # trace
np.einsum("i,j->ij", a, b)       # outer product
np.einsum("bij,bjk->bik", A_stack, B_stack) # BATCHED matmul
(B pairs)
np.einsum("bi,bi->b", x, y)      # per-row inner product
(dot batch)
```

Never `inv(A) @ b` — numerically unstable + slower than `solve(A, b)`. Use `solve`; for many RHS with same A use `scipy.linalg.lu_factor + lu_solve`.

◆ Views vs copies · dtype · memory

```
a.dtype, a.shape, a.ndim, a.size, a.itemsize, a.nbytes
a.flags.owndata          # True → owns memory; False → view
b = a[1:3]; b.base is a  # True → b is a view of a
a.astype(np.float32, copy=False)  # in-place if dtype matches
```

```
# Memory layout
a.flags["C_CONTIGUOUS"]  # row-major (numpy default)
np.ascontiguousarray(a)  # force C-contig (may copy)
```

```
# Downcast for size / speed
a.astype(np.float32)      # halves float memory vs float64
a.astype(np.int16)        # if values fit
```

```
# Common dtypes
np.float32, np.float64, np.float16
np.int8, int16, int32, int64
np.uint8 (images), np.uint32
np.bool_ (1 byte, not 1 bit)
np.complex64, complex128
```

NumPy 2.0 change: Python float/int in `np.array([...])` now infers `np.float64/np.intp` (platform-dependent int). Cast explicitly for cross-platform stability.

◆ `scipy.stats` — distributions + tests

Distributions — frozen > kwargs

```
from scipy import stats
```

```
# Frozen distribution (recommended): reusable
d = stats.norm(loc=0, scale=1)
d.pdf(0); d.cdf(1.96); d.ppf(0.975); d.rvs(size=100,
random_state=rng)
```

```
# Common distributions
stats.norm; stats.t; stats.chi2; stats.f; stats.gamma
stats.lognorm; stats.beta; stats.expon; stats.uniform
stats.poisson; stats.binom; stats.geom
stats.multivariate_normal(mean=mu, cov=S)          # multi-dim
```

Descriptive stats

```
stats.skew(x); stats.kurtosis(x, fisher=True)      # fisher=True → EXCESS kurtosis
stats.describe(x)                                  # NamedTuple summary
stats.iqr(x); stats.zscore(x, axis=0)
stats.pearsonr(x, y); stats.spearmanr(x, y); stats.kendalltau(x, y)
```

Hypothesis tests

Test	Purpose
ttest_1samp / ind / rel	mean comparison
mannwhitneyu	non-parametric 2-sample
wilcoxon	paired non-parametric
ks_1samp / ks_2samp	distribution match
shapiro / anderson	normality
chi2_contingency	categorical independence
anova (f_oneway)	k-group means

```
res = stats.ttest_ind(a, b, equal_var=False)
print(res.statistic, res.pvalue, res.df,
res.confidence_interval())
```

▲ `scipy.optimize`

```
from scipy import optimize
```

```
# Root-finding (scalar) — Brent's method: robust when you have a bracket
res = optimize.brentq(f=lambda x: np.exp(x) - 5, a=0, b=5)
```

```
# Minimise (scalar-input)
res = optimize.minimize_scalar(f=lambda x: (x - 3)**2,
method="brent")
```

```
# Minimise (vector-input)
res = optimize.minimize(
    fun=lambda w: -log_likelihood(w),
    x0=np.zeros(5),
    method="L-BFGS-B",          # default good for smooth +
    bounds
    jac=grad_fn,                # analytic gradient if
    available (else finite diff)
    bounds=[(0, 1)] * 5,
    options={"maxiter": 200})
```

```
# Constrained: SLSQP handles ineq + eq constraints
res = optimize.minimize(fun, x0, method="SLSQP",
    constraints=[{"type": "eq", "fun": lambda w: np.sum(w) - 1}],
    {"type": "ineq", "fun": lambda w: w})
```

```
# Curve fitting (non-linear least squares)
popt, pcov = optimize.curve_fit(model, x, y, p0=[1.0, 0.0])
# popt = params; pcov = covariance matrix → sd from
np.sqrt(np.diag(pcov))
```

```
# Batched LSQ
res = optimize.least_squares(residual_fn, x0, jac="3-point",
method="trf")
```

method	Use when
L-BFGS-B	smooth + box bounds; default
trust-constr	smooth + general constraints
SLSQP	constraints, small/med dim
Nelder-Mead	derivative-free, noisy fn
trf (least_squares)	NLS with bounds

◆ `scipy.sparse` (large mostly-zero matrices)

```
from scipy import sparse
# scipy ≥1.14 recommends _array over _matrix (matrix classes
deprecating).
```

```
# Build (choose format by workload)
csr = sparse.csr_array(dense)          # fast row slicing,
arith; USE for ML
csc = sparse.csc_array(dense)          # fast column slicing
coo = sparse.coo_array((vals, (rows, cols)), shape=(m, n)) #
build then convert
lil = sparse.lil_array((m, n))         # incremental build
(slow for arith)
```

```
# Convert / info
csr.toarray(); csr.shape, csr.nnz, csr.dtype
```

```
# Arithmetic — SPARSE @ DENSE stays fast
y = csr @ x                            # sparse mat-vec
A = csr.T @ csr                        # ATA — Gram
```

```
# Sparse linalg (never toarray → inv)
from scipy.sparse.linalg import spsolve, eigsh, svds, cg
x = spsolve(A csr, b)                  # direct sparse solve
vals, vecs = eigsh(A, k=6, which="LM") # symmetric eigs (Lanczos)
```

Format	Best for
CSR	row slicing, matmul, general ML
CSC	column slicing / column ops
COO	construction only; convert to CSR
LIL / DOK	incremental building

◆ `scipy` · integrate / interpolate / special

```
from scipy import integrate, interpolate, special
```

```
# 1D numerical integration
val, err = integrate.quad(lambda x: np.exp(-x**2), 0, np.inf)
```

```
# ODE solve (initial value problem)
sol = integrate.solve_ivp(fun=lambda t, y: -0.5 * y, t_span=(0, 10),
y0=[1.0], t_eval=np.linspace(0, 10,
```

```
100),
method="RK45")
```

```
# Interpolation
f = interpolate.interp1d(x, y, kind="cubic",
fill_value="extrapolate")
f_new = f(x_new)
# Modern (scipy 1.10+): CubicSpline for smoother
cs = interpolate.CubicSpline(x, y)
```

```
# Special functions (used in stats + finance)
special.erf(x); special.gamma(x); special.beta(a, b)
special.digamma(x); special.expit(x)          # sigmoid; safer than 1/(1+exp(-x))
```

▲ Top traps (6-month memory)

- **`np.random.seed()`** legacy global; use `rng = np.random.default_rng(seed)`.
- **Missing axis** — `x.sum()` collapses ALL axes; almost always want `axis=0/1`.
- **View vs copy** — basic slice = view; fancy/boolean = copy. Check `b.base is a`.
- **`inv(A) @ b`** numerically bad. Use `np.linalg.solve(A, b)`.
- **Float precision:** `a == b` unsafe. `np.isclose(a, b, rtol=1e-5)`.
- **NaN propagation:** `x.mean()` → NaN if any NaN. Use `np.nanmean`.
- **`x.std(ddof=1)`** for SAMPLE std (default `ddof=0` = population).
- **`np.empty`** = uninitialised garbage; fill before read.
- **Broadcasting** aligns FROM THE RIGHT. Use `keepdims=True`.
- **Concat in a loop = $O(N^2)$** . Build list, one final `np.concatenate`.
- **Sparse:** never `toarray()` a huge matrix. Stay in sparse space.
- **NumPy 2.0 dtype:** Python floats → `float64`; ints → `intp`. Cast explicitly for cross-platform.
- **`curve_fit` without `p0`:** silent bad local min. Always give a starting guess.

◆ Quick-reference (recall in 5 seconds)

Task	Canonical pattern
Seeded RNG	<code>rng = np.random.default_rng(42)</code>
Gaussian sample	<code>rng.standard_normal(size=(n, k))</code>
Int sample	<code>rng.integers(low, high, size, endpoint=False)</code>
Zeros	<code>np.zeros((m, n), dtype=np.float32)</code>
Range / linspace	<code>np.arange(a, b, step) / np.linspace(a, b, n)</code>
Reshape auto	<code>a.reshape(-1, k)</code>
Row-mean keeping dim	<code>x.mean(axis=0, keepdims=True)</code>
Standardise	<code>(x - x.mean(0)) / x.std(0, ddof=1)</code>
Where (ternary)	<code>np.where(cond, a, b)</code>
Multi-way select	<code>np.select([m1, m2], [v1, v2], default=0)</code>
Concat rows	<code>np.concatenate([a, b], axis=0)</code>
New axis	<code>a[:, None]</code> or <code>a[np.newaxis, :]</code>
Solve linear	<code>np.linalg.solve(A, b)</code>
SVD (economy)	<code>U, s, Vt = np.linalg.svd(A, full_matrices=False)</code>
Cholesky	<code>L = np.linalg.cholesky(A)</code>
Symmetric eig	<code>w, V = np.linalg.eigh(A)</code>
Near-equal	<code>np.isclose(a, b, rtol=1e-5)</code>
Copy	<code>a.copy()</code>
scipy dist	<code>stats.norm(loc, scale).ppf(0.975)</code>
2-sample t-test	<code>stats.ttest_ind(a, b, equal_var=False)</code>
Curve fit	<code>optimize.curve_fit(fn, x, y, p0=[...])</code>
Constrained min	<code>optimize.minimize(fn, x0, method="SLSQP", constraints=[...])</code>
Sparse solve	<code>scipy.sparse.linalg.spsolve(A_csr, b)</code>
Quadrature	<code>scipy.integrate.quad(fn, a, b)</code>